👑 CODERKING STUDIO

Official Technical Roadmaps & Architecture Guides

DATA SCIENCE & AI MATH

Matrix Multiplication, Tensors & GPU Tensor Cores

How matrix dot products power neural network forward passes and how NVIDIA Tensor Cores execute fused multiply-accumulate (FMA) in 1 clock cycle.

1. Neural Forward Pass Matrix Math

Every deep learning layer is fundamentally a matrix transformation: `Y = Activation(X · W + B)`. Batch size B, input features M, output neurons N.

```
[B × M] dot [M × N] = [B × N] Output Tensor
```

2. PyTorch GPU Tensor Acceleration

Executing matrix operations in CUDA VRAM with mixed precision:

```
import torch
A = torch.randn(4096, 4096, device='cuda', dtype=torch.bfloat16)
B = torch.randn(4096, 4096, device='cuda', dtype=torch.bfloat16)
C = torch.matmul(A, B) # 100x faster than CPU!
```

3. NVIDIA Tensor Cores Architecture

Traditional CUDA cores perform 1 FP32 calculation per cycle. Tensor Cores execute 4×4 matrix tile Fused Multiply-Accumulate in a single clock:

```
D = A × B + C (16 FP16 Multiplies + 16 FP32 Accumulates)
```

- Hopper H100: 4th Gen FP8 Tensor Cores
- 3x higher throughput for Transformer LLMs

4. Memory Bandwidth vs Compute Bound

Why LLM training stalls: SRAM cache is 100x faster than GPU HBM3 RAM. Algorithms like FlashAttention fuse matrix kernels to prevent memory bottlenecking.

- Arithmetic Intensity = FLOPs / Memory Access
- Keep weights in GPU SRAM during attention